Ders 2 Genişletilmiş Notu: Abstract Factory, Factory Method ve Builder Kalıpları

Ders 2 Genişletilmiş Notu: Abstract Factory, Factory Method ve Builder Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Kısa Tekrar ve Yaratımsal Kalıplara Giriş
- 2. Abstract Factory: Amaç, Örnek ve Çözüm
- 3. UML Gösterimi Üzerine Düzeltmeler
- 4. Switch-Case Seçeneği, Kalıp Yapısı ve Uygulanabileceği Durumlar
- 5. Abstract Factory Uygulama Örneği: Üç Boyutlu Oyun Motoru
- 6. Çıkarılan Dersler: Tasarıma Ait Genel Kurallar
- 7. Yanlış Kurulmuş Bir Kalıtım Hiyerarşisi
- 8. Factory Method Kalıbı
- 9. Factory Method Uygulama Örneği: E-Ticaret Siparişleri
- 10. Builder Kalıbı
- 11. Builder Uygulama Örneği: Çok Markalı Araç Bayisi
- Örnek Sorular
- Dersin Toparlaması

1. Kısa Tekrar ve Yaratımsal Kalıplara Giriş

Önceki derste dersin içeriği tanıtılmış, yazılım kalitesi ve iyi bir tasarıma götüren etkenler biraz daha nesnel bir biçimde ele alınmıştı. Ardından tasarım kalıplarına giriş yapılmış ve ilk kalıp olarak Model-View-Controller (MVC) görülmüştü. MVC genel mimari açısından bir öneri olduğu için ayrı tutulmuştu. Bu dersten itibaren daha ayrıntılı kalıplara geçiliyor ve Dörtlü Çete'nin (Gang of Four, GoF) kitabındaki sıra izleniyor. Bu sırada ilk grup yaratımsal (creational) kalıplardır ve bu grubun ilk kalıbı Abstract Factory'dir.

Hoca, kalıp adlarıyla ilgili basit bir ipucu verdi: bir kalıbın adında "factory" kelimesi geçiyorsa o kalıp nesne üretmeye yönelik bir kalıptır. Ama işin ayrıntıları kalıptan kalıba değişir.

2. Abstract Factory: Amaç, Örnek ve Çözüm

2.1 Amaç

Abstract Factory kalıbının amacı, birbiriyle ilişkili ya da aynı ailede olan nesneleri, somut sınıflarını belirtmeden oluşturabilmektir. Hoca bu cümlenin ilk bakışta tuhaf geldiğini kabul etti: normalde bir nesne oluşturulurken kurucu (constructor) metodu açıkça belirli bir sınıftan çağrılır. Bunu mümkün kılan, kalıtımın getirdiği çok biçimlilik (polymorphism) özelliğidir. Ancak kalıtım dikkatli kullanılması gereken bir araçtır; yanlış kullanımı çok ters sonuçlara götürebilir. Bu, dersin örneklerinde de görülecektir.

2.2 Örnek problem

GoF kitabındaki örnek bir kullanıcı arayüzü takımıdır. Java'da arayüz bileşenlerinin görünüşü ve davranışı (look and feel) çalışma anında değiştirilebilir. Linux dağıtımlarında da farklı pencere yöneticileri kullanılabilir: GNOME, KDE, daha eskilerden Motif ve Presentation Manager. Uygulama bunlardan herhangi biriyle çalışmak isteyebilir. Sorun şudur: taşınabilirliği sağlamak için istemcinin, hangi görünüşün hangi sınıftan nesnelerle gerçeklendiğini bilmemesi gerekir.

Uygulama pencere, kaydırma çubuğu (scroll bar), radyo düğmesi, onay kutusu, düğme gibi her türlü arayüz bileşenini çizmek ve kullanmak isteyecektir. Presentation Manager kullanılıyorsa onun `Window` ve `ScrollBar` sınıfları, Motif kullanılıyorsa `MotifWindow`, `MotifScrollBar`, KDE kullanılıyorsa KDE'ye özgü sınıflar kullanılacaktır. Ortada bir sürü sınıf dolaşmaya başlar ve bu çeşitlilik çalışma anında esnek davranmayı engeller.

2.3 Çözüm

Hangi pencere yöneticisi için çalışılıyorsa onun ailesinden nesneleri tek tek oluşturmak yerine olay bir kademe soyutlanır. Her arayüz bileşeni için soyut bir üst sınıf oluşturulur: soyut bir `Window`, soyut bir `ScrollBar`. Böylece kalıtımın getirdiği çok biçimlilik sayesinde istemci yalnız bu soyut üst sınıfları görür, somut sınıfları bilmek zorunda kalmaz.

Ama bu yetmez, çünkü bu nesnelerin nereden alınacağı sorusu kalır. Burada kalıbın soyut `WidgetFactory` sınıfı devreye girer. Bu soyut sınıfın her desteklenen pencere yöneticisi için bir somut alt sınıfı vardır ve her tür arayüz bileşenini oluşturan metotlar bu alt sınıflarda somut olarak gerçeklenir. Slayttaki şema şöyledir (ekrandan, 00:10:00):

```
WidgetFactory (soyut)            Client
  CreateScrollBar()                -> WidgetFactory, Window, ScrollBar ile ilişkili
  CreateWindow()
    ^ (kalıtım)
    |-- MotifWidgetFactory: CreateScrollBar(), CreateWindow()
    |-- PMWidgetFactory:    CreateScrollBar(), CreateWindow()

Window (soyut)     <- PMWindow, MotifWindow
ScrollBar (soyut)  <- PMScrollBar, MotifScrollBar

MotifWidgetFactory - - -> MotifWindow, MotifScrollBar   (kesik çizgi: oluşturur)
PMWidgetFactory    - - -> PMWindow, PMScrollBar
```

İstemciye `WidgetFactory` gerçeklemesi olarak bir `MotifWidgetFactory` nesnesi verildiğini düşünelim. İstemci pencere istediğinde `CreateWindow()` metodunu `WidgetFactory` üzerinden çağırır. Çalışan metot `MotifWidgetFactory`'nin somut `CreateWindow()` metodudur ve dönen nesne `MotifWindow` sınıfındandır. `MotifWindow`, `Window`'un alt sınıfı olduğu için istemci onu soyut üst sınıf üzerinden kullanır. Hoca bunu "tam bağımsız değil ama daha az bağımlı" diye niteledi: en özelleşmiş sınıf türü belirtilmeden belirli bir türden nesne oluşturulabilmektedir.

Hoca iki kullanım biçimini de karşılaştırdı. Pencere yöneticisi örneğinde, belirli bir dağıtım için uygulama hazırlanır ve dağıtıma uygun fabrika verilerek kodlama kolaylığı sağlanır. Java'nın look and feel örneğinde ise çalışma anında bütün bileşenlerin görünüşünü bir anda değiştirmek gibi daha ileri bir yetenek söz konusudur. Kalıp her iki durumu da destekler.

3. UML Gösterimi Üzerine Düzeltmeler

3.1 Kalıtım okunun doğru çizimi

Hoca bu noktada özellikle vurgulamak istediği bir konuya geçti. Dörtlü Çete kitabını yayımladığında UML standardı henüz oturmamıştı; farklı gösterimler bir araya getirilip son halini daha sonra aldı. Bu yüzden kitaptaki şekillerde kalıtım ilişkisi eski bir biçimde gösterilir: ok ters yöndedir ve şekil bir kalıtım üçgeninden çok varlık-ilişki (ER) diyagramlarındaki "kaz ayağına" benzer. Doğru çizimde ok ucu (içi boş üçgen) üst sınıfa bakar.

> **Hocanın vurgusu:** Ders kapsamında UML sınıf şemalarına hakimiyet ve doğru kullanım beklenir. Hoca GoF kitabından aldığı resimleri güncel gösterime çevirmedi, ama kendi örneklerindeki şemalar doğru standarda göre çizilmiştir. Sınavlarda şemalar kesinlikle yeni gösterime göre çizilmelidir.

3.2 Soyut sınıf ve soyut metot gösterimi

`WidgetFactory` sınıfının adı italik yazılmıştır, çünkü soyut bir sınıftır. İtalik yazım güncel UML'de de soyutluğu gösterir, ama elle, kâğıt üstünde çizerken belli olmaz. Bunun yerine sınıf adının üstüne iki küçüktür ve iki büyüktür işareti arasında `<<abstract>>` stereotipi yazılabilir. Aynı biçimde `CreateScrollBar()` gibi soyut bir metodun imzasının sonuna kıvrık parantez içinde `{abstract}` yazılır. Hoca bunu Violet UML aracında gösterdi: arayüzler `<<interface>>` stereotipiyle, onları gerçekleyen sınıflar ise kesik çizgili gerçekleme okuyla çizilir. Stereotip karakterleri aslında tek karakterdir (« ve »); bulunamazsa yan yana iki küçüktür ve iki büyüktür işareti kullanılabilir.

UML sınıf şemasında + public, - private anlamına gelir. Hoca, nesnelerin public metotlar üzerinden kullanıldığını, bu yüzden metotların genelde public, üye alanların ise veri gizleme ilkesine uygun olarak private yapıldığını hatırlattı. İleride Template Method gibi kalıplarda görüleceği gibi, tutarlılık için kritik olan ve değiştirilmesi istenmeyen bazı metotlar farklı erişim belirteçleriyle tanımlanabilir.

3.3 Soru-cevap: kesik çizgiler

Soru: UML gösterimindeki kesik çizgiler için de bir düzeltme yapılacak mı, yoksa sınavda o şekilde mi çizilmeli?

Cevap: Kesik çizgi kullanma (dependency) ilişkisidir. Örneğin A sınıfında M türünden bir üye alan varsa A'dan M'ye sahiplik ilişkisi çizilir. A'da X türünden bir üye alan yoksa, ama bir metodunun parametresinde ya da dönüş türünde X geçiyorsa, A'dan X'e kesik çizgili, ucu açık bir ok çizilir. GoF şemasındaki kesik çizgiler hem anlam hem çizim açısından doğrudur: somut fabrikalar somut ürünleri dönüş türü olarak üretir. İstemciden soyut sınıflara giden düz çizgiler de doğrudur, çünkü istemci oluşturduğu bileşenlerin bir listesini ya da tablosunu tutmak zorunda kalacaktır; yani gerçekten bir sahiplik ilişkisi vardır. Düzeltilmesi gerekenler yalnız kalıtım oku, kâğıt için daha uygun olan stereotip ve `{abstract}` yazımı ve metotlardaki + işaretleridir.

4. Switch-Case Seçeneği, Kalıp Yapısı ve Uygulanabileceği Durumlar

4.1 İlk akla gelen çözüm: switch-case

Aynı problem için akla ilk gelen tasarım bir switch-case olabilir. Örneğin look and feel'i ayarlayan bir metot parametre olarak bir enum değeri alır (1 ise Motif, 2 ise KDE, 3 ise GNOME gibi) ve ilgili case'te uygun nesneler oluşturulur. Bu tasarımın zayıf yönü, yeni bir pencere yöneticisi çıktığında hem enum'a yeni bir değer hem de switch'e yeni bir case eklemenin gerekmesidir.

> **Hocanın vurgusu:** Kod yazarken kendinizi switch-case kullanırken görüyorsanız, büyük ihtimalle orada kalıtımı ve kalıtımın getirdiği çok biçimlilik yeteneğini kullanan daha iyi bir tasarım yapmanız gerekir. Ayrıca switch-case içeren metotlar, benzer işler her durum için ayrı ayrı kodlandığından çok uzar.

4.2 Kalıp yapısı

Kalıbın genel yapısı, spesifik örneğin şemasının soyutlanmış halidir (ekrandan, 00:25:40):

```
AbstractFactory (soyut)                    Client
  CreateProductA()                           -> AbstractFactory
  CreateProductB()                           -> AbstractProductA
    ^                                        -> AbstractProductB
    |-- ConcreteFactory1: CreateProductA(), CreateProductB()
    |-- ConcreteFactory2: CreateProductA(), CreateProductB()
```

```
AbstractProductA (soyut) <- ProductA1, ProductA2
AbstractProductB (soyut) <- ProductB1, ProductB2

ConcreteFactory1 - - -> ProductA1, ProductB1
ConcreteFactory2 - - -> ProductA2, ProductB2
```

Her soyut fabrika somut bir fabrika tarafından gerçeklenir. Şemada dikkat edilirse soyut fabrikada üye alan yoktur. Bu yüzden `AbstractFactory` soyut sınıf olmak yerine bir arayüz (interface) olabilir. Belirli bir problem için yapılan tasarım üye alan gerektiriyorsa soyut sınıf kullanılır.

Hocanın vurgusu: Üye alana gerek yoksa her zaman arayüz kullanmayı tercih edin.

Kalıp özetle şunu söyler: bir soyut fabrika hiyerarşisi ve bir soyut ürün hiyerarşisi olsun, istemci yalnız bu soyut hiyerarşiler üzerinden ilişkilendirilsin. Bu hiyerarşilerin somut bileşenleri verildiğinde istemci doğrudan onlarla çalışmaya başlar; bunlar değiştirildiğinde de istemci duraksamadan yeni aile üyeleriyle çalışmayı sürdürür. `ProductA` ve `ProductB` ürün türleridir (örneğin pencere ve kaydırma çubuğu); `A1` ve `A2` aynı ürünün farklı ailelerdeki üyeleridir (örneğin GNOME penceresi ve KDE penceresi).

4.3 Kalıbın uygulanabileceği durumlar

Hoca, doğru kalıbı seçmenin hem sahada hem sınavlarda kritik olduğunu söyledi. Slayttaki maddeleri GoF'un ifadelerinden çevirip kendi deneyimini de eklemiştir. Abstract Factory şu durumlarda kullanılabilir (ekrandan, 00:30:00):

- Bir sistemin, kendisine ait ürünlerin nasıl oluşturulduğundan, birleştirildiğinden ve sunulduğundan bağımsız olması istendiğinde.
- Bir sistemin farklı ailelerden herhangi birindeki nesneler tarafından yapılandırılması gerektiğinde.
- İlişkili ürün ailelerinden oluşan nesnelerin birlikte kullanılması gerektiğinde.
- Bir ürün ailesindeki nesnelerin bir sınıf kütüphanesi olarak sunulacağı, ancak gerçeklemeleri yerine yalnızca arayüzlerinin açığa çıkarılacağı durumlarda.

Hoca bu maddelerin kelime kelime ezberlenmemesini istedi. Konuşma dili doğası gereği belirsizlik taşır; düşünce tarzları birbirine yakın olmayan kişiler aynı ifadeyi farklı anlayabilir. Yazılım evlerinde kullanıcıların anlattığı gereksinimlerin son haline gelene kadar ne kadar değiştiği buna örnektir. Önemli olan kafada genel bir fikrin oluşmasıdır.

5. Abstract Factory Uygulama Örneği: Üç Boyutlu Oyun Motoru

Bu örneğin kaynağı hocanın kendisidir. Bir üç boyutlu oyun motoru hazırlanmaktadır. Oyun, üzerinde çalışacağı işletim sistemine ve ekran kartına göre OpenGL ya da DirectX kütüphanesini kullanacaktır. Motorun bileşenleri arasında renderer (poligonları uygun biçimde bir araya getirerek sahneyi çizen bileşen) ve shader (ışık kaynaklarının yerine göre gölgeleri çizen bileşen) vardır; üç boyutlu grafikte akla gelebilecek başka bileşenler de olabilir. Bazı işletim sistemleri ya da ekran kartları bazı kütüphaneleri desteklemez. Örneğin Linux üstünde DirectX kullanılamaz, OpenGL kullanılabilir. Windows'ta ikisi de kullanılabilir, ama bilgisayardaki kart hangisinde daha iyi performans veriyorsa onu kullanmak en doğrusudur.

Örnek kodlar iskelet kod olarak verilir; rendering ya da shading işinin gerçekte nasıl yapıldığını kodlamak konuyu dağıtır. Odak kalıptadır.

5.1 Çözüm 1: Switch-case kullanımı

Akla ilk gelen, "quick and dirty" diye düşünülebilecek çözüm switch-case'tir. Oyun motorunu kullanan ana uygulama `ApControl` (application control) sınıfıdır. Bu sınıfta hangi kütüphaneyle çalışılacağını tutan bir `library` üye alanı ve her somut bileşen için ayrı bir üye alan vardır: OpenGL renderer için `or`, OpenGL shader için `os`, DirectX renderer için `dr`, DirectX shader için `ds`. Kurucuda `library` doğru biçimde ilklendirilir. `doRendering` metodunda `library` üzerinde switch yapılır: OpenGL ise A, B ve C rendering işlemleri `or` üzerinden, DirectX ise `dr` üzerinden yaptırılır. Shading için de aynı biçimde ayrı bir switch vardır. Diğer sınıflar şöyledir (ekrandan, 00:40:00):

```
public enum LibraryType {
    OpenGL, DirectX;
}
public class OpenGLRenderer {
    public void renderOpA() { }
    public void renderOpB() { }
    public void renderOpC() { }
}
public class OpenGLShader {
    public void shadeOpA() { }
    public void shadeOpB() { }
    public void shadeOpC() { }
}
public class DirectXRenderer {
    public void renderOpA() { }
    public void renderOpB() { }
    public void renderOpC() { }
}
public class DirectXShader {
    public void shadeOpA() { }
    public void shadeOpB() { }
    public void shadeOpC() { }
}
```

UML şemasında `ApControl`'den dört sınıfın her birine sahiplik ilişkisi vardır, çünkü her birinin türünden bir üye alanı bulunur.

5.2 Çözüm 1'in zayıf yönleri

Slayttaki tespit şudur: yeni bir kütüphane eklemek için (örneğin DirectX 10, 11 ve 12 için ayrı kütüphaneler) çok fazla yerde kod değişikliği gerekir (ekrandan, 00:52:30). DirectX 12 eklenecek olsun. `LibraryType` enum'una yeni değer yazılır; `DirectX12Renderer` ve `DirectX12Shader` sınıfları ve bunların türünden üye alanlar eklenir; `doRendering` içindeki switch'e ve `doShading` içindeki switch'e birer case eklenir. Kodun birçok farklı yerine dokunulmuş olur.

Üstelik gerçek bir motorda işlemler rendering ve shading ile sınırlı kalmaz; örnek yalın tutmak için ikisiyle sınırlanmıştır. Ray tracing (gölgelemeden farklı bir iş) ya da texture mapping gibi bir işlem eklendiğinde her aile için yeni sınıflar ve yeni üye alanlar gerekir. Yeni bir aile eklendiğinde işlem sayısı kadar yeni sınıf, yeni bir işlem eklendiğinde ise aile sayısı kadar yeni sınıf yazmak gerekir.

Hoca, ufak bir değişiklik için kodun birçok yerine dokunmayı gerektiren bu durumun ileride kod kusurları kısmında "shotgun surgery" ve "divergent change" adlarıyla görüleceğini söyledi. Bu, yazılımın bakım kolaylığının olmadığı anlamına gelir.

> **Ek bilgi (derste anlatılmadı):** Fowler'ın tanımlarında iki kusur büyük olasılıkla şöyle ayrılır: divergent change, tek bir sınıfın farklı nedenlerle sürekli değiştirilmesidir; shotgun surgery ise tek bir değişikliğin birçok sınıfta küçük düzeltmeler gerektirmesidir. Oyun motoru örneğinde yeni bir kütüphane eklemek hem `ApControl` içinde birçok yere hem de yeni sınıflara dokunmayı gerektirdiği için iki kusurun belirtileri birlikte görülür.

Hedef, değişikliklere karşı daha esnek kod yazmaktır. Ama bunu abartmamak gerekir: gelecekteki bütün değişiklikler şimdiden öngörülemez. Olması gerekenden fazla esnek yapmaya çalışmak da bir zarardır. En mantıklısı, kısa vadede olabilecek değişiklikleri göz önünde bulundurarak tasarım yapmaktır.

Slayttaki tasarım ipucu da şudur: switch-case kullanımı, çok biçimliliğe gereksinim duyulduğunun ve/veya sorumlulukların atanmasında yanlışlık olduğunun uyarı işareti olabilir (ekrandan, 00:52:30).

5.3 Çözüm 2: Kalıtımın doğru kullanımı

İkinci çözümde henüz kalıp kullanılmaz; kalıtım düzgün ve uygun biçimde kullanılır. Yapılacak ana işler (shading, rendering, ileride texture mapping) için soyut sınıflar oluşturulur, spesifik işlemler bu soyut sınıfların metotlarına eklenir. Her soyut sınıfın altında, desteklenen kütüphaneye göre somut alt sınıflar oluşturulur ve işlemlerin gövdeleri bunlarda kodlanır. Ana uygulama soyut üst sınıflar üzerinden çalışır; çalışma anında hangi türden shader ve renderer verilmişse onunla çalışır (ekrandan, 00:50:00):

```
package dp.abstractFactory.solution2;
public class ApControl {
    private LibraryType library;
    private Renderer r;
    private Shader s;
    //library nesnesini ilklendirip kütüphane belirle
    public ApControl( ) {
        switch (library){
            case OpenGL:
                r = new OpenGLRenderer();
                s = new OpenGLShader();
                break;
            case DirectX:
                r = new DirectXRenderer();
                s = new DirectXShader();
                break;
        }
    }
    void doRendering ( ) {
        r.renderOpA(); r.renderOpB(); r.renderOpC();
    }
    void doShading ( ) {
        s.shadeOpA(); s.shadeOpB(); s.shadeOpC();
    }
}
```

Şemada `<<Abstract>> Shader` (metotları `+ void shadeOpA() {abstract}` vb.) altında `OpenGLShader` ve `DirectXShader`, `<<Abstract>> Renderer` altında `OpenGLRenderer` ve `DirectXRenderer` vardır. Hoca şemadaki metot sonlarına yanlışlıkla noktalı virgül konduğunu, sınıf şemasında metotların kapanan parantezle bittiğini belirtti.

Artık dört ayrı üye alan yoktur; bir renderer ve bir shader tanımlamak yeter. Switch yalnız kurucuda kalmıştır; `doRendering` ve `doShading` içinde switch yoktur. Texture mapping eklenseydi onun gövdesinde de switch olmayacaktı.

Hoca burada önemli bir noktanın altını çizdi: tasarım kalıpları ya da daha iyi tasarımlar switch-case'i tamamen ortadan kaldırmayabilir. Amaç onu "önemli yerlerden" (where it matters) kaldırmaktır. Uygulamaya OpenGL bileşenlerinin mi DirectX bileşenlerinin mi verileceği sorusu bir yerde bir kez yanıtlanmak zorundadır. Bu karar çalışma anında otomatik da verilmeyebilir: bazı oyunlarda ana uygulamadan önce bir kurulum (setup) uygulaması ya da başlatıcı (launcher) çalışır ve kütüphane seçimi orada yapılır. Ama ilk aşamada bir yerde bir switch kalır.

Yeni bir kütüphane (DirectX 12) eklemek bu tasarımda yalnız iki yeni kardeş sınıf eklemektir: `DX12Shader` ve `DX12Renderer` (ekrandan, 00:50:00). Mevcut kodda değişiklik yapılmaz, yalnız yeni kod eklenir. Bu önemli bir avantajdır: dağıtılmış koda müdahale etmeden, mevcudu bozmadan yalnız eklemelerle daha basit bir güncelleme sahaya konabilir. Tasarım kalıbı kullanılmasa bile düzgün bir kalıtım hiyerarşisi çok daha iyi bir tasarım sağlar.

5.4 Çözüm 3: Abstract Factory kalıbının kullanımı

Üçüncü çözüm ikinci çözümün neredeyse tamamını kullanır ve buna bir fabrika ekler. Soyut `ComponentFactory` sınıfında soyut `createRenderer()` ve `createShader()` metotları vardır; bunlar soyut üst sınıflar türünden nesneler döndürür. Somut gerçeklemeleri `OpenGLFactory` ve `DirectXFactory` sınıflarıdır. `ApControl` bir renderer istediğinde ilişkilendirildiği fabrikanın

`createRenderer()` metodunu çağırır ve fabrikanın somut türüne uygun bir renderer alır; sonra onu sorunsuzca kullanır.

Kaynak kodda `library` alanı artık bir enum değil, `ComponentFactory` türündedir. Bu önemli bir farktır. Yine bir renderer ve bir shader vardır. Kurucuda kütüphane bir şekilde (yine switch ile olabilir) belirlenir; sonra renderer için `createRenderer()`, shader için `createShader()` kullanılır ve işlemler, nesnelerin DirectX mi OpenGL mi olduğuna hiç bakılmadan çalıştırılır.

DirectX 12 eklenecek olursa yine `DX12Shader` ve `DX12Renderer` kardeş sınıfları yazılır; ek olarak bir `DX12Factory` somut sınıfı da kodlanır. Kalıp kullanılınca devreye bir fabrika hiyerarşisi girmiştir.

Hoca kurucuyla ilgili bir çeşitleme daha gösterdi: kütüphane belirleme işi kurucuda yapılmak zorunda değildir, fabrika kurucuya parametre olarak verilebilir (“sen şu sisteme göre çalış”). Bu durumda o nesnenin oluşturulduğu yerde bir kez switch gerekir, başka hiçbir yerde gerekmez. Renderer ve shader’ı ayrı üye alanlar olarak tutmak yerine yalnız fabrikayı üye olarak tutup, işlem yapılacağı zaman fabrikadan ilgili nesneyi istemek de mümkündür. Hoca bunun ileride görülecek bir refactoring eylemini çağrıştırdığını söyledi.

Hoca bu kodda fabrikanın `r` ve `s` üyelerinin public olmasının doğru olmadığını, `factory.getR().renderOpA()` gibi bir erişimin daha mantıklı olacağını not etti.

5.5 Çözümlerin karşılaştırılması

Hocanın değerlendirmesi şudur: bu örnekte tasarım kalıbını kullanmak çok da gerekmiyor. Kalıtımı doğru kullanmak, en kötü haline göre çok şey kazandırıyor. Kalıp ise yalnız küçük bir ek kazanç getiriyor; bu da daha çok ana uygulamanın yapılandırma konusunda rahat etmesidir. Gereksinim değişikliğinin maliyeti normal kalıtımlı çözümde de Abstract Factory’li çözümde de aynı düzeydedir. Buna karşılık kalıp yeni sınıflar ve yeni bağlaşımlar oluşturur; bu yüzden tercih edilmeyebilir de. Hocaya göre switch-case’li çözüm kullanılmamalı, diğer ikisinden hangisi seçilirse uygundur. Ders kapsamında ise kalıbın kullanımının da gösterilmesi gerekir.

6. Çıkarılan Dersler: Tasarıma Ait Genel Kurallar

Slaytın başlığı “Çıkartılan dersler = tasarıma ait genel kurallar”dır (ekrandan, 01:00:00 ve 01:20:00).

6.1 Neyin değiştiğini bul ve onu sarmala

Sarmalamak, değişen şeyi bir sınıfla, hatta bir kalıtım hiyerarşisi içinde modellemek demektir. Bu sayede:

- Değişen şey tasarımın geri kalanından soyutlanır.
- Kara kutu yaklaşımıyla, yani public metotları üzerinden kullanılabilir.
- O konuyla ilgili gereksinim değişiklikleri yalnız o kısmı etkiler.

Önceki derste söylenen ilkeyle de bağlantılıdır: problem hangi düzeyde parçalara ayrılabiliyorsa tasarım en az o düzeyde parçalara ayrılmalıdır. Örnekte değişen şey hangi grafik kütüphanesinin kullanılacağıydı: bir bilgisayardan başka bir bilgisayara geçilince değişiyordu. Bu değişim `ComponentFactory` sınıfında soyutlandı; fabrika değiştirilerek farklı sistemlere hizmet verilebilir hale gelindi.

6.2 Kalıtım ilişkisi yerine parça-bütün ilişkisini tercih et

Deneyimli tasarımcılar genelde şöyle davranır: bir çözüm kalıtımla da kalıtımsız da yapılabiliyorsa kalıtımsız olanı tercih ederler. Hoca bunun yanlış anlaşılmaması gerektiğini söyledi: kalıtım çok güçlü bir araçtır ve olması gereken yerde mutlaka kullanılmalıdır. Kalıtım hiyerarşisi içermeyen tasarım kalıplarının sayısı bir elin parmakları kadardır; hoca örnek olarak Memento, Facade ve Mediator’ı andı.

> **Ders sonrası not:** Hoca Mediator’ı kalıtım hiyerarşisi içermeyen kalıplar arasında saydı. GoF kitabındaki Mediator yapı şemasında büyük olasılıkla soyut `Mediator` ve

`Colleague` sınıfları ile bunların somut alt sınıfları bulunur; hocanın ifadesi büyük olasılıkla kendi örnek gerçeklemelerine dayanır.

Kalıtımın sakıncası şudur: parça-bütün ilişkisinin aksine kalıtım sınıf düzeyinde kurulan, kalıcı ve çalışma anında değiştirilemeyen bir ilişkidir. Parça-bütün ilişkisi istenildiği gibi kırılıp yeni bir nesne atanarak değiştirilebilir. Bir alt sınıf nesnesini kardeş, alt ya da üst sınıftan bir nesneye çevirmek gerektiğinde bu doğrudan yapılamaz; eskisinin kopya kurucusu çağrılarak yepyeni bir nesne oluşturmak gerekir ve bu da fazla kod yazmaya götürür. İkinci olarak kalıtım kullanılırken çok dikkatli olmak ve hiyerarşiyi doğru kurmak gerekir (bkz. bölüm 7).

6.3 Gerçeklemelere göre değil, arayüzlere göre tasarla

Bu kural kara kutu yaklaşımının kullanılabilmesini sağlar. Örnekte `ApControl`, `OpenGLRenderer`, `OpenGLShader`, `DirectXRenderer`, `DirectXShader` gibi somut gerçeklemelerle doğrudan ilişki kurmadı. Soyut üst sınıfın metotları olduğunu bildi ve o metotlar üzerinden erişti. Bu, genel ve soyut bir erişim penceresi sağlar; onun yerine geçebilecek herhangi bir somut gerçekleme verildiğinde şeffaflık sağlanır. `ApControl`, fabrikadan (ya da kalıtımlı çözümde `library` nesnesinden) bir renderer ve bir shader ister; bunların nasıl oluşturulduğunu, arka planda neyle ilişkili olduklarını bilmesi gerekmez. Bu da olası en fazla esnekliği getirir.

Bu noktada hoca, Abstract Factory'yi daha önce bilen ve bir yerde kullanmış ya da benzerini adını bilmeden yapmış olanların deneyimlerini paylaşmalarını istedi; paylaşım olmadı.

7. Yanlış Kurulmuş Bir Kalıtım Hiyerarşisi

Hoca, kalıtımın farklı ve sakıncalı bir biçimde de kurulabileceğini Violet UML'de çizerek gösterdi (ekrandan, 01:30:00). Doğru çözümde değişen şey (renderer, shader) alt sınıflara konmuştu. Bunun yerine kalıtım `ApControl` üzerinden yürütülebilirdi:

```
<<abstract>> ApControl
  ^
  |-- OpenGL_ApControl    --> OpenGLShader, OpenGLRenderer (, OpenGLTextureMapper)
  |-- DirectX_ApControl   --> DirectXShader, DirectXRenderer
  |-- DirectX12_ApControl --> DirectX12Shader, DirectX12Renderer
```

Bu tasarımda birçok yerde tekrarlanan kod (duplicate code) olur. Kardeş sınıflar arasındaki tek fark, işlerin OpenGL nesnelerine mi DirectX nesnelerine mi yaptırıldığıdır. Oyunun geri kalan mantığı çerçevesindeki bütün işlemler her alt sınıfta ayrı ayrı ve boş yere tekrar kodlanır. Bu önemli bir bakım sorunudur: bu kodda en ufak bir hata ya da gereksinim değişikliği olduğunda aynı işi öteki sınıflarda da yapmak gerekir. DirectX 12 desteği istendiğinde yepyeni bir `DirectX12_ApControl` ve onun shader ile renderer'ı oluşturulur. Texture mapping eklenmek istendiğinde de her alt sınıf için ayrı bir texture mapper sınıfı gerekir. Doğru kurulmuş kalıtım çözümüne göre çok daha fazla iş çıkar.

Slayttaki özet: kalıtım `ApControl` üst sınıfından `OpenGLApControl` ve `DirectXApControl` türetilerek sakıncalı biçimde kurulabilirdi; sonuçta benzer kodun birden fazla yerde tekrarlanması nedeniyle kusurlu bir tasarım ortaya çıkardı (ekrandan, 01:20:00).

> **Hocanın vurgusu:** Kalıtım hiyerarşisini kurarken dikkatli olun. Değişen şeyi hiyerarşiye koyun, farklı bir hiyerarşi kurmaya gitmeyin. Tasarım kalıplarıyla bulunan çözüm ilk akla gelen çözüme göre biraz daha fazla iş gerektirse bile bu iş çok kısa zamanda fazlasıyla amorti olur.

8. Factory Method Kalıbı

8.1 Benzer kalıplar ve adlandırma üzerine iki uyarı

Factory Method'a geçmeden önce hoca iki uyarı yaptı. Birincisi: bazı kalıplar adları, amaçları, yaptıkları iş ya da UML sınıf şemaları açısından birbirine benzer, hatta neredeyse aynı görünebilir. Ama kesinlikle aynı değildirler; onları ayıran önemli ayrıntılar vardır. İhtiyaca en uygun kalıbı seçebilmek için bu farkları bilmek gerekir. Sınavlarda genelde "şu kalıbın şu kalıptan farkı nedir" diye sorulmasa da sorulabilir; sorulmasa bile doğru kalıp kararını verebilmek için benzer kalıplar arasındaki farklara hakim olmak gerekir.

İkincisi: bazen bir soruya “bu problemin çözümünde factory kalıbını kullanırım” diye yanıt veriliyor. Factory diye bir kalıp yoktur. Abstract Factory kalıbı ve Factory Method kalıbı vardır; “factory” nesne üretmeye yönelik metotlara verilen genel addır. Hangi kalıbın kullanılacağı açıkça belirtilmelidir.

8.2 Amaç ve örnek problem

Factory Method’un amacı, nesne oluşturmak için öyle bir arayüz sunmaktır ki oluşturulacak nesnenin sınıfına bu arayüzü sağlayan sınıfın alt sınıfları karar verebilsin. Hoca bu cümlenin kalabalık ve teknik olduğunu kabul edip örneğe geçti.

GoF’un örneklerinin çoğu çatı (framework) yazılımları üzerindendir. Çatı hazırlamak kolay değildir: çatıyı kullananlar onu ilk başta akla gelmeyecek farklı alanlarda ve amaçlarla kullanabilir. Bu yüzden esnek ve yetenekli bir tasarım gerekir; tasarım kalıplarının en çok yardım ettiği yerlerden biri budur.

Örnek: farklı belge türleriyle çalışabilecek uygulamalar geliştirmeyi kolaylaştıracak bir çatı hazırlanacaktır. Temelde belgeler ve uygulamalar vardır. Çatının kullanıcısı olan programcı, bu iki sınıftan kalıtımla yeni sınıflar türeterek kendi özelleşmiş yazılımını hazırlayacaktır. Sorun şudur: bir belge oluşturmak gerektiğinde (dosya aç, yeni dosya gibi komutlarla) çatı bu işi nasıl yapabilir? Kök neden, çatıyı yazarken kullanıcıların üreteceği yeni ve farklı belge sınıfları hakkında önceden bilgi sahibi olunamamasıdır. Kelime işlemci, hesap tablosu, iki ya da üç boyutlu çizim gibi bazı türler kestirilebilir, ama örneğin tekrar ve alt tekrar parametreleri olan bir egzersiz belgesi ilk başta akla gelmez.

8.3 Örnek çözüm: Application ve Document

Slayttaki şema şöyledir (ekrandan, 01:40:00). Hoca şemanın ortasına bir çizgi çekti: çizginin üstü çatı (framework), altı kullanıcının hazırladığı özel uygulamadır.

```
--- çatı (framework) ---
Document (soyut)                  Application (soyut)
  Open()    (soyut)    <>--docs--  CreateDocument() (soyut)
  Close()   (soyut)                NewDocument()   -> Document* doc = CreateDocument();
  Save()                                              docs.Add(doc);
  Revert()                                            doc->Open();
                                   OpenDocument()
--- özel uygulama ---
MyDocument  <- - - - - - - - - - - MyApplication
                                     CreateDocument() -> return new MyDocument
```

`Application` ve `Document` soyut sınıflardır, ama yalnız soyut metot içermezler; somut metotları da vardır. Bu yüzden burada arayüz kullanılamaz, mutlaka soyut sınıf gerekir. Elmas parça-bütün ilişkisidir; elmas bütün (`Application`) tarafındadır. Uygulama tarafının niceliği bir, belge tarafının niceliği yıldızdır: bir uygulamada n adet belge etkin olarak kullanılır.

`Document`’ın `Open()` ve `Close()` metotları soyuttur, çünkü bu işlerin nasıl yapılacağı belge türüne göre çok farklıdır ve çatı bunu öngöremez. Ama belge türü ne olursa olsun kaydedilmesi (`Save()`) ve ters bir işlem yapıldığında geri dönülmesi (`Revert()`; hocaya göre “undo” denseydi daha iyi olurdu) gerekecektir. Bunlar somut verilebilen metotlardır.

Somut metotların gövdeleri incelendiğinde içlerinde soyut metotlara çağrılar olduğu görülür. Örneğin yeni belge aç denildiğinde uygulama, belge türünden bağımsız olarak bir belge nesnesi oluşturup belgeleri tuttuğu yapıya (liste, map ya da başka bir yapı) ekler ve boş olsa da yeni belgeyi açar. Bu işler her belge türü için aynı olduğundan `NewDocument()` somuttur; belgeyi oluşturma işini ise soyut `CreateDocument()`’e bırakır. `OpenDocument()` da benzer biçimde `CreateDocument()`’ten yararlanır.

Kullanıcı kendi belge türü için `Application`’dan kalıtımla `MyApplication`’ı türetir ve `CreateDocument()`’i somut olarak yazar: belgeyi oluşturmak için gereken bilgiler alınır ve oluşturulan belge geri döndürülür. `MyDocument` de `Document`’tan türetilip soyut metotların gövdeleri doldurulur.

8.4 Bir seçenek: generic sınıflar

Bu hedefe ulaşmak için yalnız bu kalıp kullanılmak zorunda değildir. Java'da generic sınıfları bilenler, oluşturulacak belge türünden elemanlar içeren generic bir belge sınıfı yapmayı düşünebilir. Bu yapılabilir ve elde edilen çözüm bazı avantajlara da sahip olabilir. Ama zayıf yanı Java diline özgü olmasıdır. C++'ta template sınıflar, C#'ta benzer generic yapılar vardır, ama ayrıntılar farklıdır; bazı dillerdeki ayrıntılar işi kolaylaştırır, bazılarındakiler zorlaştırır. Hoca, taşınabilirliği azalttığı ve ek iş gerektirdiği için dile özgü çözümler üzerinden konuşmayı tercih etmediğini söyledi.

8.5 Kalıp yapısı

Kalıbın genel yapısı şöyledir (ekrandan, 01:46:00):

```
Product (soyut)                Creator (soyut)
  ^                              FactoryMethod() (soyut)
  |                              AnOperation()    -> ... product = FactoryMethod() ...
ConcreteProduct  <- - - - - -  ConcreteCreator
                                 FactoryMethod()  -> return new ConcreteProduct
```

Bir oluşturucu (`Creator`) sınıfı ve onun belirli bir türden ürün oluşturan metodu vardır; kalıba adını veren bu factory method'dur. `Creator` soyuttur ve factory method üst sınıfta soyut bir metottur. `Creator`'ın ürünle birlikte yapacağı her türlü işlem bulunabilir ve bunların en azından bir kısmı somuttur. Ne kadar çok somut işlem konabilirse çatı o kadar yetenekli olur ve kullanıcıların yükü o kadar azalır. Bu somut metotların içinde uygun anlarda factory method çağrılır. `ConcreteCreator` ve `ConcreteProduct` kalıtımla oluşturulur; `ConcreteCreator`, `ConcreteProduct` türünden nesneleri oluşturup döndürdüğü için aralarında kullanma (bağımlılık) ilişkisi vardır. Örnekteki uygulama-belge parça-bütün ilişkisi kalıbın zorunlu bir parçası değildir; ihtiyaca göre olabilir.

8.6 Kalıbın uygulanabileceği durumlar ve paralel hiyerarşiler

Factory Method şu durumlarda kullanılabilir:

- Yeni nesneler oluşturması gereken bir sınıf, oluşturacağı nesnelerin spesifik türünü bilemiyorsa.
- Bir sınıf kendi üzerindeki bir sorumluluğu çeşitli yardımcı alt sınıflara devrediyorsa ve hangi sorumluluk için hangi tür yardımcının oluşturulacağına karar vermek gerekiyorsa. Burada switch-case akla gelebilir, ama bunun iyi bir çözüm olmadığı hatırlatıldı.

İkinci durumu somutlaştıran örnek bir çizim uygulamasıdır (ekrandan, 01:50:00). Şekiller fareyle değiştirilebilecek (manipüle edilebilecek), bu yüzden hem figürler hem de düzenleyiciler (manipülatörler) olacaktır:

```
Client --> Figure, Manipulator

Figure (soyut)                      Manipulator (soyut)
  CreateManipulator() (soyut)         DownClick(), Drag(), UpClick()  (soyut)
  ^                                   ^
  |-- LineFigure: CreateManipulator()   |-- LineManipulator: DownClick(), Drag(), UpClick()
  |-- TextFigure: CreateManipulator()   |-- TextManipulator: DownClick(), Drag(), UpClick()
  |-- ...                               |-- ...

LineFigure - - -> LineManipulator,  TextFigure - - -> TextManipulator
```

Bu örnekteki factory method `CreateManipulator()`'dır. Çizgi, metin, dikdörtgen, üçgen gibi farklı figürler için farklı manipülatörler kullanılır. Bir çizgide başlangıç ve bitiş noktaları değiştirilebilir, bu da açıyı ve uzunluğu değiştirir; düz, kesikli, noktalı çizgi biçimleri desteklenebilir. Metin figüründe ise yeni harf eklemek, kalın ya da italik yapmak gibi işler vardır. Bu yüzden farklı figürler farklı manipülatörlerle ilişkilendirilmek zorundadır.

Manipülatörün soyut işlemleri fare olaylarıdır: düğmeye basma (`DownClick`), sürükleme (`Drag`) ve düğmeyi bırakma (`UpClick`). Hangi bileşen manipüle edilirse edilsin bu olayların dinlenip

kodlanması gerekir; ne yapılacağı figüre göre değiştiği için soyut bırakılırlar. Gerekirse somut metotlar da olabilir.

Hoca kalıbın desteklediği çeşitlemeleri de saydı. Bazı figürler ortak bir manipülatörle düzenlenebilir. Bir nesne birden fazla manipülatörle de idare edilebilir; örneğin taşıma ve döndürme için farklı manipülatörler kullanılabilir. Bazı şekiller kendine özel bir manipülatöre ihtiyaç duymayabilir; bu durumda varsayılan bir manipülatör (`DefaultManipulator`) kardeş sınıf olarak eklenebilir ya da üstteki soyut sınıf somut yapılıp varsayılan olarak kullanılabilir. Üst sınıf somut yapılırsa, alt sınıfları oluştururken mutlaka olması gereken metotları ezmeyi (override) unutmamak gerekir; üst sınıf soyutken derleyici bu konuda uyarı verir, somut olunca bu güvence kaybolur.

9. Factory Method Uygulama Örneği: E-Ticaret Siparişleri

9.1 Problem

Bu örnek Head First Design Patterns kitabından alınmıştır. Bir e-ticaret sitesi farklı tür ürünler satmaktadır. Site çalışanı bir müşterinin siparişindeki farklı tür ürünleri işleme koyacaktır. Sipariş işleme adımları belli bir sırada yürütülür, ama adımlardaki iş mantığı ürün türüne göre çok farklıdır (ekrandan, 02:00:00). Örneğin gıda gibi hızlı tüketilen bir ürün doğrudan satılabilir; elektronik bir ürün için garanti süreçleri nedeniyle ayrı kayıt ve raporlama işleri gerekir; kırılabilecek bir eşya için paketlemeye daha fazla dolgu malzemesi konması gerekir. Hoca burada ufak tefek farklardan değil, çok farklı biçimde yürütülen adımlardan söz ettiğini vurguladı. Site çalışanına, sipariş vermede kullanabileceği bir araç sağlanacaktır. Kullanım şöyle olabilir:

```
package factoryMethod.example;
public class MainApp {
    public static void main(String[] args) {
        Store store;
        //Elektronik ürün siparişi geldi
        store = new ElectronicsStore( );
        store.createOrder("5524345678");
        //Kitap siparişi geldi
        store = new BookStore( );
        store.createOrder("8693243565");
    }
}
```

Farklı dükkan türleri farklı ürünler oluşturup kullandığı için, oluşturulacak ürünlerin oluşturulmasına yönelik bir kalıp olarak Factory Method uygun görünmektedir.

> **Ders sonrası not:** Kullanım örneğinde `store.createOrder(...)` çağrılıyor. Çözümde `createOrder` protected ve soyut bir metottur, siparişin adımlarını yürüten ise public `makeOrder` metodudur (bkz. 9.2). Kullanım örneğinde büyük olasılıkla `makeOrder` çağrılması amaçlanmıştır; `createOrder` yalnız sipariş nesnesini oluşturur, adımları yürütmez.

9.2 Çözüm

Hoca bu şemaların görünümünün farklı olduğunu, bunları koddan otomatik şema çıkaran bir Eclipse eklentisiyle çizdiğini anlattı. Eclipse eklentileri çoğu zaman belirli bir Eclipse sürümüne özgüdür; eklentiyi geliştiren ekip dağılırsa ya da yeni eklenti mimarisine uyum sağlayamazsa eklenti atıl kalır. Bu eklentinin bir sorunu da koddaki bütün ilişki türlerini şemaya almaya çalışması ve şemaları çok kalabalık yapmasıydı; iki sınıf arasında belirli bir ilişkiyi gösterip ötekini gizlemek gibi ince ayarlar yapılamıyordu. Hoca bu yüzden sonra Violet UML'ye geçtiğini söyledi.

Çözümün şeması şöyledir (ekrandan, 02:04:30):

```
Order (soyut) [Product]                  Store (soyut) [Creator]
  + orderFromSupplier()  (soyut)           # createOrder(type: String): Order  (soyut)
  + pack()               (soyut)           + makeOrder()
```

```
  + prepare()            (soyut)
  + sendToCustomer()     (soyut)
  ^                                        ^
  |-- Electronics [Concrete Product]       |-- ElectronicsStore [Concrete Creator] --<<instant
  |     + Electronics(String), ...         |-- BookStore        [Concrete Creator] --<<instant
  |-- Books [Concrete Product]
        + Books(String), ...
```

`Store`'un `createOrder` metodu protected yapılmıştır; somut alt sınıfların da alt sınıfları olursa o metoda ulaşabilsinler diye. Hoca bunun şart olmadığını, ama bir şey kaybettirmediğini söyledi. `Order` sınıfındaki işlemlerin hepsi soyuttur. Problem tanımında farklı tür siparişlerin işlemlerinde çok büyük farklar olduğu söylenmişti; bu yüzden ortak bir davranışı üst sınıfa kodlamak mümkün değildir. Örneğin `Electronics`'in kurucusu ürünü tanımlayan bir kimlik (barkod) alır; tedarikçiden sipariş, paketleme, hazırlama ve müşteriye gönderme adımları ürüne göre kodlanır. Elektronikte sigorta gerekebilir, kitapta gerekmez; çocuk kitaplarıyla yetişkin kitaplarının vergileri farklı olabilir.

`Store`'un `createOrder` metodu soyuttur, çünkü farklı siparişler çok farklı biçimlerde oluşturulur. `makeOrder` ise kendi içinde belirli bir adım sırası olduğu için somuttur: önce siparişi oluşturur, sonra `Order`'ın mutlaka yapılması gereken işlerini sırayla çağırır (ekrandan, 02:07:40):

```
public abstract class Store {
    protected abstract Order createOrder( String uniqueID );
    public void makeOrder( String uniqueID ) {
        Order newOrder = createOrder( uniqueID );
        newOrder.prepare( );
        newOrder.orderFromSupplier( );
        newOrder.pack( );
        newOrder.sendToCustomer( );
    }
}
public class ElectronicsStore extends Store {
    protected Order createOrder( String uniqueID ) {
        Electronics newElectronicsOrder = new Electronics(uniqueID);
        //Elektronik malzeme siparişi için yapılacak işlemlerin tanımlanması
        return newElectronicsOrder;
    }
}
public class BookStore extends Store {
    protected Order createOrder(String uniqueID ) {
        Books newBookOrder = new Books(uniqueID);
        //Kitap siparişi için yapılacak işlemlerin tanımlanması
        return newBookOrder;
    }
}
```

`ElectronicsStore` yalnız `createOrder` metodunun gövdesini tanımlar: yeni bir elektronik siparişi oluşturur, gerekli işleri ekler ve siparişi döndürür. `BookStore` da aynı biçimde kitaba özgü işlerle çalışır. Böylece "şu tip dükkanı oluştur, oradan şu siparişi ver" biçiminde ortak bir çalışma tarzı elde edilir. Hoca bu örnekte ayırt edici bir karşılaştırma gerekmediğini; ileride aynı problemi farklı yöntemlerle çözüp kaynak kodları karşılaştıracakları kalıplar olacağını söyledi.

10. Builder Kalıbı

10.1 Amaç ve örnek problem

Builder kalıbının amacı, karmaşık bir nesnenin oluşturulmasını gösteriminden ayırmak, soyutlamaktır. Böylece aynı oluşturma işlemi farklı gösterimler ortaya çıkarmak için kullanılabilir.

GoF örneği bir metin okuyucudur. Okuyucu RTF (Rich Text Format) biçiminde metin okuyacak ve belgeyi farklı metin türlerine çevirebilecektir: düz metin, TeX, hatta metni düzenleyebilecek bir arayüz bileşeni (widget). Sorun şudur: çok farklı metin türleri vardır ve olabildiğince çoğu

desteklenmek istenir, ama sahaya çıkarken hepsini gerçeklemek düşünülmez. Önce acil ihtiyaç olan türler yapılır, kalanlar ileride eklenir. Bu yüzden önemli bir ölçüt vardır: tasarıma ileride yeni bir metin türü eklemek zor olmamalıdır.

Yine "neyin değiştiğini bul ve sarmala" ilkesi uygulanır: oluşturma ve gösterim birbirinden ayrışıyorsa oluşturma ayrı, gösterim ayrı yapılır. Hoca burada MVC'nin akla gelebileceğini, ama o kalıbın odağının veriyi nasıl oluşturacağımız değil, veriyi farklı biçimlerde gösterebilmek ve denetleyebilmek olduğunu belirtti.

10.2 Örnek çözüm: RTF okuyucu

Çözümde metni okumakla sorumlu bir sınıf, çevirme isteğini bir biçim çevirici nesnesine gönderir. Çevirici nesne hem çevirme işleminden hem de çevrilen metni belirli bir biçimde (düz metin, TeX vb.) temsil etmekten sorumludur. Yeni metin türleri için çevirici sınıfın yeni alt sınıflarını oluşturmak yeter (ekrandan, 02:30:00):

```
RTFReader                  <>--builder-->  TextConverter (soyut)
  ParseRTF()                                   ConvertCharacter(char)    (soyut)
                                               ConvertFontChange(Font)   (soyut)
                                               ConvertParagraph()        (soyut)
                                               ^
   |-- ASCIIConverter:      ConvertCharacter(char), GetASCIIText()        - - -> ASCIIText
   |-- TeXConverter:        ConvertCharacter(char), ConvertFontChange(Font),
   |                        ConvertParagraph(), GetTeXText()              - - -> TeXText
   |-- TextWidgetConverter: ConvertCharacter(char), ConvertFontChange(Font),
                            ConvertParagraph(), GetTextWidget()           - - -> TextWidget
```

```
ParseRTF() notu:
while (t = get the next token) {
    switch t.Type {
    CHAR:
        builder->ConvertCharacter(t.Char)
    FONT:
        builder->ConvertFontChange(t.Font)
    PARA:
        builder->ConvertParagraph()
    }
}
```

Çevirici metindeki her bileşeni tek tek çevirir: karakterin çevrilmesi, font değişikliğinin çevrilmesi, paragrafın çevrilmesi farklıdır; başka bileşenler için ek metotlar eklenebilir. `TextConverter` ve metotları soyuttur, çünkü bu işlerin yapılma biçimi değişir. ASCII çevirici yalnız karakterleri çevirir: düz metinde font yoktur, paragraf da yalnız bir yeni satır karakteridir; Word'deki satır aralığı, önceki ve sonraki boşluk gibi paragraf biçimlemeleri düz metinde yoktur. TeX çeviricide ise karakter, font ve paragraf çevrimlerinin hepsinin anlamı vardır; çevirme bitince sonuç `GetTeXText()` ile alınır. Metin bileşeni çevirici de benzer işleri yapar ve bir widget verir. Çıktılar (`ASCIIText` vb.) şemada kesik çizgiyle gösterilmiştir; hoca bunları nesne olarak değil, o türde metin içerikleri olarak düşünmeyi önerdi.

Metin okunurken ayrıştırılır (parse): açılan dosyadaki her token sırayla alınır; karakterse builder'dan karakteri çevirmesi, font bilgisiyse font değişikliğini ele alması, paragrafsa paragrafı çevirmesi istenir.

10.3 Çözümün eleştirileri

Slayttaki iki eleştiri şudur (ekrandan, 02:30:00):

- Çok biçimlilik tam kullanılamadı, switch-case hâlâ duruyor. Hoca burada switch'ten kaçılamadığını söyledi: belge biçiminde farklı türde bileşenler vardır ve her token okunduğunda türüne bakıp işlemek gerekir. Bunun bir sakıncası yoktur, ama bir eleştiri olarak söylenebilir.

- Alt sınıflar üst sınıfın tüm metotlarını gerçeklemelidir; bu yüzden alt sınıfta anlamı olmayan metotlar boş gövdeli olarak kalmak zorundadır. Hocaya göre asıl büyük eleştiri budur. `ASCIIConverter` font ve paragraf değişikliğinin anlamını bilmez, ama üst sınıftan kalıtımla aldığı için bu metotların gövdelerini tanımlamak zorundadır: açılır ve kapanır kıvrık parantezlerle, boş gövde olarak.

Bu, kalıtımın çekirdek tanımına aykırıdır. Alt sınıftan bir nesne, üst sınıftan bir nesnenin daha özelleşmiş halidir: üst sınıfta ne varsa alt sınıfta da vardır ve daha fazlası olabilir. ASCII çevirici örneğinde ise üst sınıftaki bazı işlerin alt sınıfta boş kodlandığı görülür. Kod kalitesi politikaları varsa bu durum onları ihlal edebilir; böyle bir durumda kalıp kullanılmayabilir. Hoca, bu anlamsız metotları yazmayı gerektirmeyen bir nesneye yönelik dil varsa kalıbın sorunsuz kullanılabileceğini, ama C++, C#, Java gibi geleneksel dillerde boş gövdeli metotların önemli bir sorun olduğunu söyledi. Yine de böyle bir çalışma biçimine ihtiyaç varsa "her güzelin bir kusuru vardır" diyerek kalıp seçilebilir.

> **Ek bilgi (derste anlatılmadı):** GoF kitabı bu sorunu büyük olasılıkla, soyut Builder'daki build metotlarını saf soyut yapmak yerine varsayılan olarak boş gövdeli tanımlamayı önererek ele alır; böylece somut builder'lar yalnız ilgilendikleri işlemleri ezer. Bu, boş gövdeyi alt sınıflardan üst sınıfa taşır ama hocanın kalıtımın anlamına dair eleştirisini tamamen ortadan kaldırmaz.

10.4 Kalıp yapısı

Kalıbın yapısı şöyledir (ekrandan, 02:40:00):

```
Director                 <>--builder-->  Builder (soyut)
  Construct()                               BuildPart() (soyut)
    -> for all objects in structure {       ^
           builder->BuildPart()             |
       }                                  ConcreteBuilder  - - - -> Product
                                            BuildPart()
                                            GetResult()
```

`Director`, `Construct()` metodunda oluşturulacak nesnenin her bir parçasını alır ve builder'a "o parçayı yap" komutunu gönderir. Bu komutu farklı gösterimlere yol açabilecek farklı somut builder'lar yerine getirir ve ortaya ürün çıkar. Slayttaki not: kalıbın bu tasarımında switch-case görünmez, ama `ConcreteBuilder.buildPart` metodunda kendini gösterecektir; çünkü `Director.construct` metodundaki "for all objects in structure" döngüsündeki nesneler bir şekilde `buildPart` metoduna parametre olarak aktarılmak zorunda kalınacak ve "parça şu ise şunu yap, bu ise bunu yap" gibi komutlar verilmek zorunda kalınacaktır.

10.5 Sequence diyagramlarında dönüş okları

Kalıbın etkileşimleri bir sequence diyagramıyla gösterilir: istemci önce bir `ConcreteBuilder` oluşturur, sonra bunu verip bir `Director` oluşturur ve director'a `Construct` der; `Construct` içinde A, B ve C parçaları oluşturulur, sonra istemci sonucu builder'dan alır.

Hoca bu noktada GoF'un ve birçok kitabın sequence diyagramlarını fazla rahat çizdiğini eleştirdi. Bu şemalar çalışma zamanını gösterir ve onlardan kaynak kodun gerçekte nasıl çalıştığına dair bilgi çıkarılabilmelidir. Bunun için dönüş oklarının gösterilmesi gerekir. Dönüş okları çizilmezse bir metodun, diğerinin tamamlanmasından sonra mı yoksa onun içinde mi çağrıldığı bilgisi kaybolur. Şemada aşağı inildikçe zamanda ilerlenir; `Construct`'ın başlangıcı ve bitişi dönüş oklarıyla belirlenirse A, B, C parçalarının `Construct`'ın içinde oluşturulduğu açıkça görülür. Aksi halde parçaların `Construct` bittikten sonra mı oluşturulduğu anlaşılamaz. Yanıt beklenmeyen eşzamansız (asenkron) haberleşmeler bunun dışındadır, ama bir metodun çalışma ayrıntıları gösterilirken bu olmaz.

> **Hocanın vurgusu:** Sequence diyagramlarında dönüş okları mutlaka çizilmelidir.

10.6 Kalıbın uygulanabileceği durumlar ve kazanımları

Builder şu durumlarda kullanılabilir:

- Karmaşık bir nesneyi oluşturma algoritmasının, bu nesneyi oluşturan parçalardan bağımsız olması isteniyorsa ve aynı zamanda (ikinci şart da geçerli olmalı) parçaların nasıl birleştirileceğinden de bağımsız olması gerekiyorsa.
- Oluşturma sürecinin, nesnenin farklı gösterimlerine izin vermesi isteniyorsa.

Kazanım: ürünün iç yapısı en az yan etkiyle değiştirilebilir; yeni bir `ConcreteBuilder` sınıfı yazmak yeter. RTF örneğinde metnin farklı biçimde inşa edilmesi için yeni bir kardeş çevirici yazılır. Ama metinde yeni bir parça türü söz konusu olursa hem switch'e yeni bir case eklemek hem de ilgili convert metodunu eklemek ya da değiştirmek gerekir. Hocaya göre bu çok fazla bir değişiklik değildir, maliyet getirmeyen bir çözümdür.

10.7 Builder ile Abstract Factory ve Factory Method arasındaki fark

Builder'ın sonucunda da, Abstract Factory ve Factory Method'da da bir nesne oluşturulur. Algılanması gereken fark şudur: Builder kalıbında istemci (director) ürün oluşturma süreci üzerinde çok daha fazla denetim sahibidir. Parçaların nasıl birleştirildiğinin orkestrasyonunu, iş adımlarını ve sırasını o yürütür. Builder'ın convert metotlarını çağırmakla kalmayıp önüne, arkasına ekleyeceği komutlarla çok daha fazla ve değişik işler yaptırabilir. Abstract Factory ve Factory Method'da ise "şöyle bir şey lazım" denir ve ürün alınır; ürünün alt ayrıntılarına ("şurası şöyle olsun") karışılamaz. Builder'da oluşturma ayrıntılarına ince ayarla müdahale edilebilir. Bu ihtiyaç varsa Builder, yoksa duruma göre Abstract Factory ya da Factory Method kullanılır.

11. Builder Uygulama Örneği: Çok Markalı Araç Bayisi

11.1 Problem

Bu örnek de hocanın kendisine aittir (slaytta "Kaynak: YES"). Farklı marka ve modellerde araçların satıldığı bir bayi için bilgi sistemi hazırlanacaktır. Daha geniş bir pazara hitap edebilmek için tek bir markaya bağlı kalınmak istenmez. Sorun, markaların model adlarında kendilerine özgü adlandırma ifadeleri kullanmasıdır: kasa türü, otomatik vites, motor gücü gibi bilgiler model adında markaya özgü kısaltmalarla geçer ve aynı şey farklı markalarda farklı biçimde ifade edilir. Slayttaki problem tanımı şöyledir (ekrandan, 02:50:00):

- Farklı marka ve model araçların satıldığı bir bayide, markalara özel adlandırma ifadelerine rağmen ortak bir araç modellemesi yapılacak.
- Araçlar kasa, yakıt türü, motor hacmi ve gücü gibi bileşenlerin bir araya getirilmesi ile karmaşık bir süreçle oluşmaktadır.

Ürün paketi (`product`) şöyledir:

```
Car
  - brand, model : String
  --> Transmission:  - gearCount : int {final}
                     - isAutomatic : boolean {final}
  --> Engine:        - volume, power : int
                     --> <<enum>> FuelType: PETROLEUM, DIESEL, LPG, ELECTRIC
  --> <<enum>> Chassis: SEDAN, HATCHBACK, STATIONWAGON, SUV
```

11.2 Neden Abstract Factory değil?

İlk bakışta Abstract Factory işe yarar gibi görünebilir: bir arabanın kasası, yakıt türü, motor gücü vardır; Renault ya da Honda fabrikasından bunları istemek akla gelebilir. Ama o kalıpta ürün aileleri ortaktır: pencere her ailede penceredir, radyo düğmesi radyo düğmesidir. Burada ise ürün çok daha fazla sayıda alt parçadan oluşur ve bu alt parçalar markadan markaya farklı yorumlanır. Bu yüzden Builder kalıbı en uygun olanıdır.

11.3 Çözüm

`CarDirector` soyut bir `CarBuilder` nesnesiyle çalışır. İlk sürümde somut olarak Renault ve Honda modelleri desteklenir (`RenaultBuilder`, `HondaBuilder`). Builder director'a kurucusunda verilir. Director'a string bileşenlerden oluşan bir içerik verildiğinde her eleman tek tek alınır;

elemanın hangi parçayla ilgili olduğunu çözen iş mantığı director'dadır ve ilgili build metodu çağrılır. Bu, kalıbın kendisinde de olan "parça şöyleyse şunu yap" yapısıdır. Sonunda ortaya çıkan aracı rapor olarak ekrana basan bir metot vardır.

Builder'da `buildChassis`, `buildEngine`, `buildTransmission` metotları ve oluşturulan aracı vermek için `getResult` metodu vardır. Ürüne alt sınıflardan erişilebilmesi gerekir ki inşa edilebilsin; ama dışarıdan doğrudan alınması istenmez, yalnız bütün işler bittikten sonra sonucun alınması istenir. Bu yüzden ürün protected bir getter ile alt sınıflara açılır, dışarıya ise public `getResult` sunulur. `getResult` ürün null değilse ve doğru ilklendirilmişse (kasası, motoru ve vitesi ayarlanmışsa; motor ve vites için "tamam mısın" diye soran bir kontrol olabilir) ürünü verir. Ürünü doğrudan değil, kopyasını döndürür: her bilgiyi bir klona kopyalar ve klonu gönderir. Doğrudan gönderseydi istemci ürünü kafasına göre değiştirebilir ve bu değişiklik builder'ın içindeki ürünü de değiştirmiş olurdu.

Director'ın `construct` metodu her elemanı sekme karakterinden ayırır; solundaki kısım parça türünü, sağındaki kısım değeri verir (ekrandan, 02:55:00):

```
int tab = element.indexOf("\t");
String item = element.substring(0,tab);
String info = element.substring(tab+1,element.length());
if( item.compareToIgnoreCase("Engine")== 0 )
    builder.buildEngine(info);
else if( item.compareToIgnoreCase("Gear")== 0 )
    builder.buildTransmission(info);
```

Hoca kaynak kodların paylaşılan dosyada üç ana klasör halinde olduğunu anlattı: tasarım kalıbı örnekleri, ileride görülecek refactoring örnekleri ve refactoring'lerin JUnit sınamalarını içeren testing paketleri. Refactoring her zaman birim sınamaları eşliğinde yapılacaktır.

11.4 Markaya özgü yorumlama: HondaBuilder ve RenaultBuilder

`HondaBuilder`'da marka Honda olarak atanır. Kasa oluşturulurken model adında "back" geçiyorsa (fastback, speedback vb.) araç hatchback, "CR-V" geçiyorsa SUV sayılır; ikisi de değilse varsayılan olarak sedan atanır. Hoca bunların güncelliği konusunda bir iddiası olmadığını, yalnız örnek olduğunu söyledi. Motor oluşturan kod şöyledir (ekrandan, 03:00:00):

```
public void buildEngine(String token) {
    //Min. values for a Honda engine
    if( getProduct().getEngine() == null )
        getProduct().setEngine( new Engine(FuelType.PETROLEUM, 1150, 60) );
    token = token.toUpperCase();
    if( token.contains("ECO") )
        getProduct().getEngine().setFuel(FuelType.LPG);
    else if( token.contains("PS") ) {
        int end = token.indexOf("PS");
        token = token.substring(0, end);
        getProduct().getEngine().setPower( Integer.parseInt(token) );
    }
    else if( token.contains("L") ) {
        int end = token.indexOf("L");
        token = token.substring(0, end);
        getProduct().getEngine().setVolume( (int)(Double.parseDouble(token) * 1000) );
    }
}
```

Motor adında "ECO" geçiyorsa yakıt LPG'dir. "PS" geçiyorsa önündeki sayı motor gücüdür ve tamsayıya çevrilir. "L" geçiyorsa önündeki sayı litre cinsinden hacimdir ve 1000 ile çarpılarak cc'ye çevrilir; örneğin 1.2 litre 1200 cc olur. Hoca birim çevrimlerine her zaman dikkat edilmesini söyledi. Vites için de benzer bir kod vardır; kodun açıklamasına göre varsayılan otomatik vites 4, düz vites 5 viteslidir.

`RenaultBuilder`'da yorumlama farklıdır. Kasa adında station wagon'a karşılık gelen ifade geçiyorsa kasa station wagon, Koleos ise SUV, Symbol gibi modellerde sedan, aksi halde hatchback

atanır. Motor adında dizeli belirten ifade geçiyorsa yakıt dizel, aksi halde benzindir; güç ve hacim de Renault'nun adlandırma mantığına göre (hacimde "L" yazılmadan) çözülür. Hoca bu adlandırma kurallarını örnek için uydurduğunu söyledi.

11.5 Çalıştırma ve çıktı

`main` metodu şöyledir (ekrandan, 02:55:00 ve 03:03:25):

```
public static void main(String[] args) {
    CarDirector director = new CarDirector( new HondaBuilder("Civic Sedan") );
    String elements[] = {"Engine\t125PS","Engine\t1.6L","Gear\t6AT"};
    director.construct(elements);
    director.report();
}
```

Adım adım:

- Verilen: Honda builder, model metni "Civic Sedan"; elemanlar `Engine 125PS`, `Engine 1.6L`, `Gear 6AT`.
- Kasa: model adında "back" ya da "CR-V" geçmediği ve kasayla ilgili ayrı bir bilgi verilmediği için varsayılan sedan atanır.
- Motor: ilk `buildEngine` çağrısında motor null olduğundan önce asgari değerlerle (benzin, 1150 cc, 60) bir motor oluşturulur. `125PS` içinde "PS" geçer; güç 125 olur. `1.6L` içinde "L" geçer; hacim $(int)(1{,}6 \times 1000) = 1600$ cc olur.
- Vites: `6AT` 6 ileri otomatik vites olarak yorumlanır.
- Sonuç (konsol çıktısı):

```
Car: Honda Civic Sedan (Sedan)
Engine: Petroleum 1600cc 125HP
Transmission: 6 gears  automatic transmission
```

> **Ek bilgi (derste anlatılmadı):** Model metnindeki "PS" (Pferdestärke) metrik beygir gücüdür; çıktıdaki "HP" ise genelde mekanik beygir gücünü gösterir. 1 PS büyük olasılıkla yaklaşık 0,986 hp'dir; yani 125 PS yaklaşık 123 hp eder. Örnek kodda bu ayrım yapılmadan sayı doğrudan aktarılmıştır; kalıbın anlatımı açısından bir önemi yoktur.

Hoca, amacın gerçekten model adlarını çözen bir uygulama yazmak değil, karmaşık parçalardan ve yapılandırma ayrıntılarından oluşan bileşik bir nesneyi oluşturmaya yönelik Builder kalıbını incelemek olduğunu vurguladı.

Örnek Sorular

Soru 1: Kesik çizgili ok ne anlama gelir? (derste sözlü, 00:31:06)

GoF'un Abstract Factory şemasındaki kesik çizgili oklar düzeltilmeli mi? Kesik çizgili ok hangi ilişkiyi gösterir ve ne zaman kullanılır?

Derste verilen çözüm: Kesik çizgili, ucu açık ok kullanma (dependency) ilişkisidir. A sınıfında X türünden bir üye alan yoksa, ama bir metodunun parametresinde ya da dönüş türünde X geçiyorsa A'dan X'e kesik çizgili ok çizilir. A'da X türünden bir üye alan varsa sahiplik ilişkisi çizilir. GoF şemasında somut fabrikalar somut ürünleri dönüş türü olarak ürettiği için kesik çizgiler doğrudur; düzeltilmesi gereken kalıtım okunun yönü ve biçimi, soyutluğun `<<abstract>>` ve `{abstract}` ile gösterimi ve metotlardaki + işaretleridir.

Soru 2: Oyun motorunda yeni kütüphane eklemek (ekrandan, 00:40:00, 00:50:00 ve 00:52:30)

Bir üç boyutlu oyun motoru, işletim sistemine ve ekran kartına göre OpenGL ya da DirectX kütüphanesinin renderer ve shader bileşenlerini kullanacaktır. (a) Switch-case ile, (b) doğru kurulmuş kalıtımla, (c) Abstract Factory ile kurulan çözümlerde DirectX 12 desteği eklemek için neler yapılmalıdır? Hangi çözüm tercih edilmelidir?

Derste verilen çözüm:

- (a) `LibraryType` enum'una yeni değer, `DirectX12Renderer` ve `DirectX12Shader` sınıfları, bunların türünden yeni üye alanlar ve `doRendering` ile `doShading` içindeki switch'lere birer yeni case eklenir. Kodun birçok yerine dokunulur.
- (b) Soyut `Renderer` ve `Shader` altına yalnız `DX12Renderer` ve `DX12Shader` kardeş sınıfları eklenir; mevcut kod değişmez (kurucudaki tek switch dışında).
- (c) (b)'deki sınıflara ek olarak bir `DX12Factory` somut fabrikası yazılır.
- Değerlendirme: asıl kazanç doğru kalıtımdan gelir; (b) ile (c)'nin değişiklik maliyeti aynı düzeydedir. Kalıp yapılandırmayı rahatlatır ama yeni sınıflar ve bağlaşımlar getirir. Switch-case'li çözüm kullanılmamalı, diğer ikisinden biri seçilebilir.

Soru 3: Site çalışanı için sipariş aracı tasarlayın (ekrandan, 02:00:00 ve 02:04:30)

Bir e-ticaret sitesi farklı tür ürünler satıyor. Site çalışanı bir müşterinin siparişindeki farklı tür ürünleri işleme koyacak. Sipariş işleme adımları belli bir sırada yürütülüyor, ancak adımlardaki iş mantığı ürün türüne göre çok farklılık gösteriyor. Site çalışanının `store = new ElectronicsStore(); ...` biçiminde kullanabileceği bir tasarım yapın.

Derste verilen çözüm: Factory Method kalıbı kullanılır. Soyut `Store` (Creator) sınıfında protected soyut `createOrder(uniqueID): Order` factory method'u ve public somut `makeOrder(uniqueID)` metodu bulunur. `makeOrder` önce `createOrder` ile siparişi oluşturur, sonra sırasıyla `prepare()`, `orderFromSupplier()`, `pack()`, `sendToCustomer()` çağırır. Soyut `Order` (Product) sınıfındaki bu dört işlemin hepsi soyuttur, çünkü ortak bir davranış yoktur. `ElectronicsStore` ve `BookStore` (Concrete Creator) sırasıyla `Electronics` ve `Books` (Concrete Product) nesneleri oluşturur. Kod bölüm 9.2'dedir.

Soru 4: Araç bayisi için hangi kalıp? (ekrandan, 02:50:00)

Farklı marka ve model araçların satıldığı bir bayide, markalara özel adlandırma ifadelerine rağmen ortak bir araç modellemesi yapılacaktır. Araçlar kasa, yakıt türü, motor hacmi ve gücü gibi bileşenlerin bir araya getirilmesiyle karmaşık bir süreçle oluşmaktadır. Hangi kalıp kullanılmalıdır, neden Abstract Factory değildir?

Derste verilen çözüm: Builder kalıbı kullanılır. Abstract Factory'de ürün aileleri ortaktır (her ailede bir pencere, bir kaydırma çubuğu vardır); burada ise ürün çok sayıda alt parçadan oluşur ve bu parçalar markadan markaya farklı yorumlanır. `CarDirector` model metnindeki elemanları ayırıp parça türüne göre `buildEngine`, `buildTransmission` gibi metotları çağırır; `HondaBuilder` ve `RenaultBuilder` markaya özgü yorumlamayı yapar; sonuç `getResult()` ile, ürünün kopyası olarak alınır. Kod ve çıktı bölüm 11'dedir.

Soru 5: Builder ile Abstract Factory / Factory Method arasındaki fark (derste sözlü, 02:40:23)

Builder, Abstract Factory ve Factory Method kalıplarının hepsinin sonucunda bir nesne oluşturulur. Builder'ı ötekilerden ayıran fark nedir?

Derste verilen çözüm: Builder'da istemci (director) ürünün oluşturulma süreci üzerinde çok daha fazla denetim sahibidir: parçaların nasıl birleştirileceğini, adımların sırasını yönetir, builder'ın metotlarının önüne ve arkasına başka işler ekleyebilir. Abstract Factory ve Factory Method'da ise "şu lazım" denir ve ürün alınır; oluşturmanın ayrıntılarına ince ayarla müdahale edilemez. Oluşturma ayrıntılarına müdahale ihtiyacı varsa Builder, yoksa duruma göre Abstract Factory ya da Factory Method seçilir.

Dersin Toparlaması

Bu derste yaratımsal kalıpların ilk üçü işlendi. Abstract Factory, ilişkili nesne ailelerini somut sınıfları belirtmeden üretmeyi sağlar: bir soyut fabrika ve bir soyut ürün hiyerarşisi kurulur, istemci yalnız soyutlarla konuşur. Oyun motoru örneği, switch-case çözümünden doğru kalıtıma ve oradan kalıba giden yolu gösterdi; asıl kazancın doğru kalıtımdan geldiği, kalıbın ise yapılandırma rahatlığı getirdiği görüldü. Bu örnekten üç genel tasarım kuralı çıkarıldı: neyin değiştiğini bul ve sarmala, kalıtım yerine parça-bütün ilişkisini tercih et, gerçeklemelere değil arayüzlere göre tasarla. Yanlış kurulmuş bir kalıtım hiyerarşisinin kod tekrarına yol açtığı

da gösterildi. Ders boyunca UML gösterimine dair düzeltmeler yapıldı: güncel kalıtım oku, `<<abstract>>` ve `{abstract}` gösterimi, kullanma ilişkisi ve sequence diyagramlarında dönüş okları.

Factory Method, oluşturulacak nesnenin sınıfına alt sınıfların karar vermesini sağlar; çatı (framework) örneğinde ve e-ticaret sipariş örneğinde, sabit iş akışını tutan somut metotların içinden soyut bir oluşturma metodunun çağrıldığı görüldü. Builder ise karmaşık bir nesnenin oluşturulmasını gösteriminden ayırır; director parçaları sırayla builder'a yaptırır. Builder'ın zayıf yönleri (kalan switch ve boş gövdeli metotlar) ile Abstract Factory ve Factory Method'dan farkı (oluşturma süreci üzerindeki denetim) tartışıldı ve çok markalı araç bayisi örneğiyle kalıp uygulamalı olarak gösterildi. Benzer kalıplar arasındaki farkları bilmenin doğru kalıp seçimi için şart olduğu vurgulandı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.